



CONECTA-BAIRRO

| Documentação Técnica de Engenharia de Software

Projeto	Plataforma Web de Marketplace Local de Serviços
---------	---

Disciplina	Programação para Web
------------	----------------------

Tipo	Projeto de Extensão Universitária
------	-----------------------------------

Stack	Node.js - Express.js - MySQL - REST API
-------	---

Repositório	github.com/PedroPCamaral/conecta-bairro-fullstack
-------------	---

Ano	2026
-----	------

Sumário

1. Introdução e Visão Geral do Projeto

2. Levantamento de Requisitos

3. Especificação de Requisitos Funcionais e Não Funcionais

4. Metodologia Ágil Aplicada (Scrum)

5. Arquitetura de Software

6. Modelagem e Dicionário do Banco de Dados

7. Especificação da API REST

8. Design de Interface, UX/UI e Identidade Visual

9. Padrões de Codificação e Boas Práticas

10. Plano de Testes e Qualidade

11. Roadmap e Melhorias Futuras

12. Considerações Finais

1. Introdução e Visão Geral do Projeto

O **Conecta-Bairro** é uma plataforma web full stack que atua como um marketplace comunitário de serviços locais, conectando moradores da região de Águas Claras – DF a prestadores de serviços autônomos. O sistema centraliza a busca, o contato e a divulgação de serviços prestados dentro da comunidade, promovendo inclusão digital, fortalecimento da economia local e geração de oportunidades para profissionais autônomos.

Este documento consolida todo o processo de engenharia de software aplicado ao projeto: desde o levantamento de requisitos junto aos stakeholders (moradores e prestadores de serviço), passando pela definição da arquitetura cliente-servidor, modelagem do banco de dados relacional, até a especificação da API REST e os padrões de codificação adotados no desenvolvimento.

1.1 Justificativa

Águas Claras é uma das regiões de maior densidade populacional do Distrito Federal, com aproximadamente 141.872 habitantes e crescimento populacional anual estimado em 8%. Esse cenário gera forte demanda por serviços locais (elétrica, hidráulica, costura, pintura, carpintaria, tecnologia, entre outros), mas a oferta desses serviços é hoje pulverizada e informal — divulgada majoritariamente por indicações pessoais ou redes sociais pessoais dos prestadores, sem um canal único de busca e contratação.

1.2 Objetivos do Projeto

Objetivo Geral

Desenvolver uma plataforma web full stack que centralize a divulgação e a busca de prestadores de serviço locais, aplicando as boas práticas de engenharia de software, arquitetura cliente-servidor e persistência de dados relacional.

Objetivos Específicos

- Implementar API REST com operações CRUD completas;
- Modelar banco de dados relacional normalizado;
- Garantir responsividade e usabilidade da interface;
- Aplicar metodologia ágil no ciclo de desenvolvimento.

1.3 Escopo do Sistema

Dimensão	Descrição
Produto	Plataforma web (marketplace) de serviços locais, com áreas distintas para morador e prestador
Usuários-alvo	Moradores de Águas Claras – DF (buscam serviços) e prestadores de serviço autônomos (divulgam perfil profissional)
Prazo	Definido em ciclo acadêmico por sprints, conforme cronograma da disciplina
Orçamento	Projeto de extensão universitária — infraestrutura própria (hospedagem local/gratuita em fase de desenvolvimento)
Fora do escopo (versão atual)	Autenticação de usuários, pagamentos online, geolocalização, aplicativo mobile nativo

2. Levantamento de Requisitos

O levantamento de requisitos do Conecta-Bairro seguiu um conjunto combinado de técnicas — **briefing**, **entrevista semiestruturada** e **questionário** — de modo a captar tanto as necessidades de negócio quanto as expectativas de uso dos dois perfis de usuário do sistema: o morador e o prestador de serviço.

2.1 Briefing do Projeto

O briefing reúne as informações essenciais coletadas junto aos responsáveis pelo projeto antes do início do desenvolvimento, servindo como documento norteador das decisões de produto e design.

Campo	Informação levantada
Título do projeto	Conecta-Bairro — Plataforma Inteligente de Marketplace para Serviços Locais
Problema identificado	Ausência de canal centralizado para busca e divulgação de prestadores de serviço na região
Público-alvo primário	Moradores de Águas Claras – DF, todas as faixas etárias com acesso à internet
Público-alvo secundário	Prestadores de serviço autônomos das categorias: elétrica, hidráulica, costura, alimentação, limpeza, tecnologia, pintura e carpintaria
Restrições técnicas	Stack definida pela disciplina: Node.js, Express.js e MySQL
Critério de sucesso	CRUD funcional de prestadores, busca por categoria/nome operacional, interface responsiva

2.2 Roteiro de Entrevista Aplicado

Foi conduzida uma entrevista semiestruturada — com roteiro-guia, mas com liberdade para aprofundar respostas — representando a etapa em que a equipe de desenvolvimento buscou compreender o comportamento e as dores dos dois perfis de usuário.

Roteiro — Perfil Morador

- Como você atualmente encontra prestadores de serviço no seu bairro?
- Quais categorias de serviço você mais procura?
- O que mais pesa na hora de escolher um prestador (preço, avaliação, proximidade)?
- Você prefere acessar esse tipo de serviço pelo celular ou computador?
- O que tornaria a busca por um prestador mais fácil e confiável?

Roteiro — Perfil Prestador de Serviço

- Como você divulga seus serviços hoje?
- Quais dificuldades você enfrenta para ser encontrado por novos clientes?
- Quais informações você considera essenciais no seu perfil (contato, endereço, descrição)?
- Você teria interesse em manter seu próprio cadastro atualizado na plataforma?
- Que funcionalidade tornaria a plataforma mais vantajosa para o seu negócio?

Síntese das respostas coletadas

Necessidade identificada	Origem	Impacto no sistema
Buscar por categoria e por nome	Entrevista com moradores	Endpoints de filtro e busca na API
Visualizar dados de contato e localização do prestador	Entrevista com moradores	Campos phone, address, neighborhood na tabela providers
Cadastrar, editar e remover o próprio perfil	Entrevista com prestadores	Operações CRUD (POST, PUT, DELETE)
Divulgação sem custo	Entrevista com prestadores	Modelo de negócio sem cobrança na versão atual
Acesso via celular	Questionário	Requisito não funcional de responsividade

2.3 Questionário de Requisitos

Complementarmente à entrevista, aplicou-se um questionário objetivo para consolidar decisões de escopo técnico e funcional:

Questionário de Requisitos — Conecta-Bairro

- Onde o sistema será acessado? *Navegador web, desktop e mobile.*
- Quem irá acessar? *Moradores (consulta) e prestadores (cadastro/gestão do próprio perfil).*
- Quando será utilizado? *Sempre que houver necessidade de contratar um serviço local.*
- Como o sistema atenderá à necessidade? *Por meio de busca, filtro por categoria e listagem de prestadores ativos.*
- O que acontece ao cadastrar um prestador? *O registro é persistido no MySQL e passa a constar nas buscas.*
- Qual o objetivo central? *Reduzir a fricção entre morador e prestador de serviço local.*

2.4 Persona e Contexto Social

Morador (Persona A)



Busca prestadores confiáveis próximos ao seu bairro

Dor: falta de canal único de busca e contato direto

Dispositivo: majoritariamente mobile

Prestador (Persona B)



Deseja mais visibilidade para o seu serviço

Dor: divulgação informal e dependente de indicação

Necessidade: gestão do próprio perfil

3. Especificação de Requisitos

3.1 Requisitos Funcionais (RF)

ID	Descrição	Prioridade
RF01	O sistema deve permitir listar todas as categorias de serviço cadastradas	Alta
RF02	O sistema deve permitir listar todos os prestadores de serviço ativos	Alta
RF03	O sistema deve permitir buscar um prestador específico por ID	Alta
RF04	O sistema deve permitir filtrar prestadores por categoria	Alta
RF05	O sistema deve permitir buscar prestadores por nome (busca textual)	Alta
RF06	O sistema deve permitir o cadastro (criação) de um novo prestador	Alta
RF07	O sistema deve permitir a atualização dos dados de um prestador existente	Média
RF08	O sistema deve permitir a remoção do cadastro de um prestador	Média
RF09	O sistema deve exibir dados de contato do prestador (telefone, e-mail, endereço, bairro)	Alta
RF10	O sistema deve marcar prestadores como ativos/inativos (campo isActive)	Média

3.2 Requisitos Não Funcionais (RNF)

ID	Descrição	Categoria
RNF01	A interface deve ser responsiva, adaptando-se a desktop, tablet e mobile	Usabilidade
RNF02	A comunicação entre frontend e backend deve ocorrer via API REST em JSON	Arquitetura
RNF03	O banco de dados deve garantir integridade referencial (FK entre providers e categories)	Confiabilidade
RNF04	O sistema deve responder às requisições de listagem em tempo aceitável para uso interativo	Desempenho
RNF05	O código deve seguir padronização de nomenclatura e ser versionado em Git/GitHub	Manutenibilidade
RNF06	O sistema deve ser executável em ambiente Node.js 14+ e MySQL 5.7+	Portabilidade
RNF07	A identidade visual deve seguir a paleta de cores institucional definida	Usabilidade

4. Metodologia Ágil Aplicada

O desenvolvimento do Conecta-Bairro adotou uma abordagem ágil baseada em **Scrum simplificado**, com apoio de quadro **Kanban** para controle visual do fluxo de tarefas, adequada ao contexto de equipe reduzida e ciclo acadêmico.

4.1 Papéis Definidos

Papel	Responsabilidade
Product Owner	Prioriza o backlog e valida entregas frente às necessidades levantadas com moradores e prestadores
Equipe de Desenvolvimento	Implementa backend, frontend e integração com o banco de dados
Professor orientador	Atua como stakeholder e revisor técnico do projeto de extensão

4.2 Backlog do Produto (User Stories)

Como...	Eu quero...	Para que...
Morador	buscar prestadores por categoria	eu encontre rapidamente o serviço que preciso
Morador	ver o contato de um prestador	eu possa entrar em contato diretamente
Prestador	cadastrar meu perfil profissional	eu ganhe visibilidade na região
Prestador	editar minhas informações	eu mantenha meus dados atualizados
Prestador	remover meu cadastro	eu possa sair da plataforma quando quiser

4.3 Quadro Kanban do Sprint

BACKLOG	A FAZER	EM ANDAMENTO	CONCLUÍDO
Sistema de avaliações	Testes automatizados da API	Filtro de busca por nome	CRUD de prestadores
Autenticação de usuários	Documentação técnica		Modelagem do banco de dados
Dashboard administrativo			Conexão Node.js + MySQL
			Listagem de categorias

4.4 Cronograma de Sprints

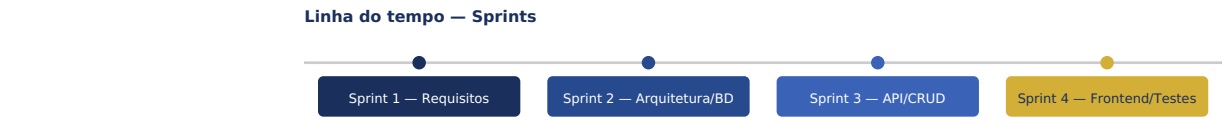


Figura 2 — Distribuição das entregas ao longo dos sprints

5. Arquitetura de Software

O Conecta-Bairro adota uma **arquitetura cliente-servidor em três camadas**, com frontend desacoplado, backend RESTful em Node.js/Express e persistência relacional em MySQL. A estrutura modular do projeto separa claramente as responsabilidades entre apresentação, regras de negócio/rotas e acesso a dados — aproximando-se dos princípios do padrão MVC.

5.1 Diagrama de Arquitetura

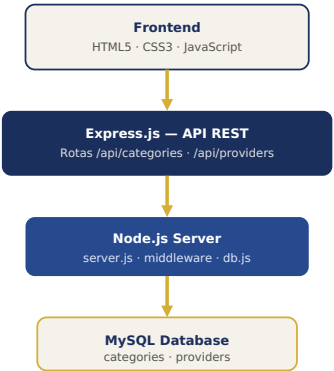


Figura 3 — Fluxo de comunicação entre as camadas da aplicação

5.2 Estrutura de Diretórios

```
conecta-bairro/
├── public/           # Frontend estático
│   ├── index.html
│   ├── style.css
│   └── script.js
├── routes/          # Definição das rotas da API REST
│   └── api.js
├── middleware/       # Middlewares Express (validação, tratamento de erros)
├── scripts/          # Scripts auxiliares (seed, manutenção do banco)
├── db.js             # Configuração da conexão com o MySQL
├── server.js         # Ponto de entrada da aplicação
├── package.json
└── README.md
```

5.3 Padrão Arquitetural

Camada	Responsabilidade	Tecnologia
Apresentação (View)	Renderização da interface e interação com o usuário	HTML5, CSS3, JavaScript Vanilla
Controle (Controller/Rotas)	Recebe requisições HTTP, valida entrada e direciona para a lógica de negócio	Express.js (routes/api.js, middleware)
Dados (Model)	Acesso e persistência de dados	MySQL via db.js (mysql2)

5.4 Fluxo de uma Requisição

Uma requisição típica — por exemplo, a busca de prestadores por categoria — percorre o seguinte caminho: (1) o frontend dispara uma chamada `fetch` para o endpoint `GET /api/providers/category/:id`; (2) o Express.js roteia a chamada para o handler correspondente em `routes/api.js`; (3) o middleware trata validações e possíveis erros; (4) o handler executa a consulta SQL via `db.js`; (5) o resultado é serializado em JSON e devolvido ao frontend, que atualiza a interface dinamicamente.

6. Modelagem e Dicionário do Banco de Dados

O banco de dados relacional `conecta_bairro` foi modelado em MySQL, contemplando duas entidades principais — **categorias** e **prestadores** — relacionadas por uma chave estrangeira, garantindo integridade referencial entre os cadastros.

6.1 Diagrama Entidade-Relacionamento (DER)

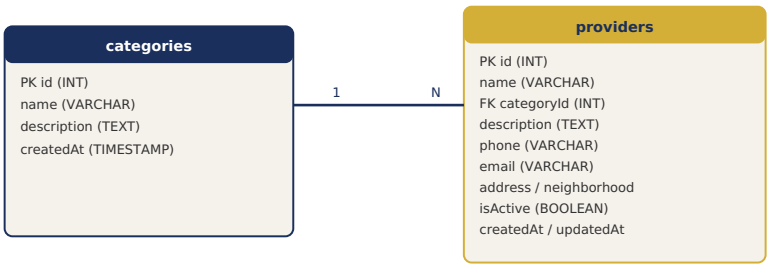


Figura 4 — DER simplificado: uma categoria possui N prestadores

6.2 Script de Criação

```
CREATE DATABASE conecta_bairro;
USE conecta_bairro;

CREATE TABLE categories (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE providers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  categoryId INT NOT NULL,
  description TEXT,
  phone VARCHAR(20),
  email VARCHAR(320),
  address VARCHAR(255),
  neighborhood VARCHAR(100),
  isActive BOOLEAN DEFAULT 1,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (categoryId) REFERENCES categories(id)
);
```

6.3 Dicionário de Dados — categories

Campo	Tipo	Restrição	Descrição
id	INT	PK, AUTO_INCREMENT	Identificador único da categoria
name	VARCHAR(100)	NOT NULL	Nome da categoria de serviço
description	TEXT	—	Descrição da categoria
createdAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Data de criação do registro

6.4 Dicionário de Dados — providers

Campo	Tipo	Restrição	Descrição
id	INT	PK, AUTO_INCREMENT	Identificador único do prestador
name	VARCHAR(255)	NOT NULL	Nome do prestador de serviço
categoryId	INT	FK → categories(id)	Categoria à qual o prestador pertence
description	TEXT	—	Descrição do serviço oferecido
phone	VARCHAR(20)	—	Telefone de contato
email	VARCHAR(320)	—	E-mail de contato
address	VARCHAR(255)	—	Endereço do prestador
neighborhood	VARCHAR(100)	—	Bairro de atuação
isActive	BOOLEAN	DEFAULT 1	Indica se o cadastro está ativo
createdAt / updatedAt	TIMESTAMP	auto-gerenciado	Datas de criação e última atualização

6.5 Categorias Seed

As categorias iniciais cadastradas para viabilizar a operação do marketplace são: Elétrica, Costura, Alimentação, Limpeza, Tecnologia, Hidráulica, Pintura e Carpintaria — refletindo diretamente as demandas identificadas na etapa de entrevista com os moradores.

7. Especificação da API REST

A API segue o estilo arquitetural REST, com respostas em JSON, verbos HTTP semânticos e recursos organizados por entidade (`categories` e `providers`).

7.1 Endpoints — Categorias

Método	Endpoint	Função
GET	/api/categories	Lista todas as categorias cadastradas

7.2 Endpoints — Prestadores

Método	Endpoint	Função
GET	/api/providers	Lista todos os prestadores ativos
GET	/api/providers/:id	Busca um prestador específico pelo ID
GET	/api/providers/category/:id	Filtra prestadores por categoria
GET	/api/providers/search/:query	Busca prestadores por nome
POST	/api/providers	Cria um novo prestador
PUT	/api/providers/:id	Atualiza os dados de um prestador
DELETE	/api/providers/:id	Remove o cadastro de um prestador

7.3 Padrão de Resposta e Tratamento de Erros

Todas as respostas da API devem seguir um formato JSON consistente, com código de status HTTP condizente com o resultado da operação, conforme convenção adotada no projeto:

Status HTTP	Situação
200 OK	Requisição processada com sucesso (GET, PUT)
201 Created	Recurso criado com sucesso (POST)
204 No Content	Remoção realizada com sucesso (DELETE)
400 Bad Request	Dados de entrada inválidos ou incompletos
404 Not Found	Recurso (prestador/categoria) não encontrado
500 Internal Server Error	Falha inesperada no servidor ou na conexão com o banco

Boa prática adotada

Os middlewares de tratamento de erros centralizam a captura de exceções (try/catch nas rotas assíncronas), evitando vazamento de stack trace ao cliente e padronizando o corpo de erro no formato `{ "error": "mensagem descritiva" }`.

8. Design de Interface, UX/UI e Identidade Visual

O design do Conecta-Bairro segue princípios de usabilidade e experiência do usuário voltados à simplicidade de navegação, priorizando a rapidez na busca por um prestador de serviço — a tarefa central do morador dentro do sistema.

8.1 Paleta de Cores Institucional

Creme Claro #F5F1EB	Azul Marinho #1A2F5C	Dourado #D4AF37

A combinação transmite confiabilidade (azul marinho), acolhimento e proximidade comunitária (creme) e destaque para ações e elementos de valor (dourado), reforçando a identidade de um marketplace local de confiança.

8.2 Princípios de UX aplicados

Princípio	Aplicação no Conecta-Bairro
Hierarquia visual	Categorias em destaque na home, seguidas da listagem de prestadores
Feedback ao usuário	Estados de carregamento e mensagens de erro/sucesso nas operações de CRUD
Consistência	Componentes reutilizados (cards de prestador) em toda a aplicação
Acessibilidade	Contraste adequado entre texto e fundo, seguindo a paleta institucional
Responsividade	Layout adaptável entre desktop, tablet e mobile

8.3 Prototipagem

O processo de design seguiu a evolução clássica de fidelidade de protótipos: **rafe de baixa fidelidade** (esboço inicial da tela principal e do card de prestador), evoluindo para **protótipo de média fidelidade** (wireframes navegáveis das telas de busca, listagem e formulário de cadastro) e, por fim, a **interface de alta fidelidade** implementada em HTML5/CSS3, já com a paleta de cores e tipografia definitivas aplicadas.

9. Padrões de Codificação e Boas Práticas

Para garantir manutenibilidade e um código livre de erros comuns, o projeto adota um conjunto de convenções de codificação aplicadas de forma consistente em todo o backend Node.js/Express.

9.1 Convenções de Nomenclatura

Elemento	Convenção	Exemplo
Variáveis e funções	camelCase	<code>getProviderById</code>
Tabelas e colunas do banco	camelCase / snake_case consistente	<code>categoryId</code> , <code>createdAt</code>
Rotas da API	substantivos no plural, kebab-case quando composto	<code>/api/providers</code>
Arquivos	lowerCamelCase ou kebab-case por módulo	<code>db.js</code> , <code>api.js</code>

9.2 Boas Práticas Adotadas

- Uso de **funções assíncronas (async/await)** com blocos try/catch em todas as rotas que acessam o banco de dados, evitando promises não tratadas;
- Uso de **prepared statements** (queries parametrizadas) no acesso ao MySQL, prevenindo injeção de SQL;
- **Middleware centralizado de tratamento de erros**, evitando duplicação de lógica de captura de exceção em cada rota;
- **Separação de responsabilidades** entre configuração de conexão (`db.js`), definição de rotas (`routes/api.js`) e ponto de entrada (`server.js`);
- Variáveis sensíveis (credenciais de banco) mantidas fora do controle de versão, via arquivo de ambiente e `.gitignore` ;
- Validação de entrada nos endpoints POST/PUT antes da persistência dos dados.

9.3 Checklist de Revisão de Código (Code Review)

- ☐ A rota trata erros de conexão/consulta ao banco?
- ☐ Os dados de entrada são validados antes da query?
- ☐ A query utiliza parâmetros (não concatenação de string)?
- ☐ O endpoint retorna o status HTTP correto?
- ☐ O nome da função/variável é autoexplicativo?
- ☐ Não há credenciais expostas no código-fonte?

10. Plano de Testes e Qualidade

10.1 Estratégia de Testes

Tipo de teste	Objetivo	Ferramenta/Método
Teste manual de API	Validar cada endpoint isoladamente	curl / Postman
Teste de integração	Validar o fluxo completo frontend → API → banco	Execução end-to-end no navegador
Teste de responsividade	Validar adaptação do layout em diferentes resoluções	DevTools (modo responsivo)
Teste de integridade de dados	Validar constraint de chave estrangeira entre providers e categories	Inserção/remoção de registros de teste no MySQL

10.2 Casos de Teste — Exemplo

ID	Cenário	Resultado esperado
CT01	GET /api/categories	Retorna 200 e lista de categorias em JSON
CT02	GET /api/providers/category/999 (categoria inexistente)	Retorna lista vazia ou 404, conforme regra definida
CT03	POST /api/providers sem campo obrigatório "name"	Retorna 400 com mensagem de validação
CT04	DELETE /api/providers/:id em ID inexistente	Retorna 404
CT05	PUT /api/providers/:id com dados válidos	Retorna 200 e dados atualizados

Comandos de teste da API

```
curl http://localhost:3000/api/providers
curl http://localhost:3000/api/categories
```

11. Roadmap e Melhorias Futuras

Melhoria	Justificativa
Sistema de autenticação	Permitir que cada prestador gerencie exclusivamente o próprio cadastro
Avaliações e comentários	Aumentar a confiança do morador na escolha do prestador
Integração com WhatsApp	Reduzir a fricção no contato direto entre as partes
Pagamentos online	Viabilizar transações completas dentro da plataforma
Geolocalização	Priorizar prestadores mais próximos do morador
Aplicativo mobile	Ampliar o alcance e a conveniência de uso
Deploy em nuvem	Disponibilizar o sistema fora do ambiente local de desenvolvimento
Dashboard administrativo	Permitir gestão centralizada de categorias e prestadores

12. Considerações Finais

O Conecta-Bairro demonstra a aplicação prática do ciclo completo de engenharia de software — do levantamento de requisitos à implementação de uma arquitetura cliente-servidor funcional — em um problema real da comunidade de Águas Claras – DF. A adoção de metodologia ágil permitiu entregas incrementais organizadas por sprints, enquanto a modelagem relacional do banco de dados garante a integridade e a escalabilidade do cadastro de categorias e prestadores.

Este documento consolida as decisões técnicas do projeto e serve como referência para a manutenção evolutiva do sistema, incluindo as melhorias futuras mapeadas no roadmap, como autenticação, avaliações e geolocalização.

"O sistema foi desenvolvido com foco em tecnologia, impacto social e inovação local — conectando moradores e prestadores de serviço de forma simples, acessível e centralizada."

Ficha Técnica

Item	Descrição
Projeto	Conecta-Bairro
Disciplina	Programação para Web
Tipo	Projeto de Extensão
Ano	2026
Licença	MIT — uso acadêmico